SYNOPSIS

ON

"VerifyRAG: Claim-Level Citation Verification for Reducing Hallucination in Retrieval-Augmented Generation"

Submitted in

Partial Fulfillment of the requirements for the Award of the Degree

of

Bachelor of Technology

In

Computer Science and Engineering

by

(Project Id:)

Vansh Gupta (2501640101006)

Under the supervision of
Mohd Shuaib Zahid
(Assistant Professor)



Pranveer Singh Institute of Technology.
Kanpur - Agra - Delhi National Highway - 19
Bhauti -Kanpur - 209305.

1. Introduction

With the rapid emergence of Generative Artificial Intelligence (GenAI) and Large Language Models (LLMs), natural language interfaces have become foundational tools for automated question answering, data synthesis, and technical reasoning. Despite their remarkable fluency, LLMs frequently suffer from hallucination—the generation of factually unsupported, extralinguistic, or contradictory assertions presented with undue statistical confidence. Retrieval-Augmented Generation (RAG) was introduced to alleviate this vulnerability by grounding text generation in external, domain-specific corpora via dense vector retrieval.

However, conventional RAG systems operate under an unverified, open-loop assumption: context passages are retrieved, the LLM produces an answer, and the synthesized response is presented directly to the user without validating whether every constituent statement is faithfully grounded in the retrieved context. In high-stakes environments—such as legal analysis, medical inquiry, financial audits, and scientific research—even minor hallucinations can yield catastrophic misinformation.

VerifyRAG proposes an automated, claim-level verification and remediation framework that fact-checks generated answers against retrieved source passages prior to presentation. The system decomposes complex responses into atomic propositions, scores each claim using a cross-encoder Natural Language Inference (NLI) model, and remediates unsupported claims via active filtering or critique-guided regeneration.

Special Technical Terms:

- **Retrieval-Augmented Generation (RAG):** A hybrid AI architecture combining non-parametric dense retrieval (vector database) with parametric generation (LLM).
- **Hallucination:** Extrinsic or intrinsic fabrications where generated text diverges from factual ground-truth source evidence.
- **Natural Language Inference (NLI):** The classification task of determining whether a hypothesis is logically Entailed, Contradicted, or Neutral given a premise text.
- **Cross-Encoder Architecture:** A deep transformer architecture that processes premise and hypothesis jointly via full cross-attention, providing superior verification precision over bi-encoder similarity.

• **Atomic Claim Decomposition:** Syntactic deconstruction of compound statements into discrete, independently testable factual propositions.

2. Project Objective

The primary objective of VerifyRAG is to construct a verifiable, transparent, and hallucination-resistant retrieval-augmented generation framework. The specific project objectives are:

1. Develop an end-to-end grounded retrieval pipeline using recursive text chunking and dense vector indexing with FAISS.

2. Implement propositional claim decomposition to split compound generated sentences into atomic, testable assertions using linguistic parsers.

3. Integrate a cross-encoder NLI model (cross-encoder/nli-deberta-v3-base) to classify each claim against retrieved evidence chunks into Entailment, Neutral, or Contradiction.

4. Implement dual post-verification remediation strategies: (a) Filtering Mode to purge or flag unverified claims, and (b) Regeneration Mode to re-synthesize responses using strictly verified premises.

5. Achieve a quantifiable reduction ($\geq 30\%$) in unsupported claims compared to baseline Plain RAG, while sustaining an acceptable verification latency overhead (< 2.5 seconds).

6. Deliver an interactive, explainable Streamlit demonstration dashboard featuring claim-level citation badges, bi-directional evidence highlighting, and real-time latency telemetry.

3. Feasibility Study

A comprehensive feasibility study was conducted to analyze the technical, operational, economic, schedule, and legal viability of the proposed VerifyRAG system:

3.1 Technical Feasibility

The project is technically feasible using current hardware and open-source deep learning frameworks. Dense retrieval is performed using Facebook AI Similarity Search (FAISS), which completes top-k vector searches in under 50ms. Cross-encoder verification is powered by the pre-trained DeBERTa-v3 architecture, renowned for state-of-the-art accuracy on NLI and fact-checking benchmarks (MNLI, FEVER). By leveraging an NVIDIA GeForce RTX 3050 GPU (4

GB VRAM) with CUDA acceleration, half-precision (FP16) tensor operations, and batched matrix inference, the cross-attention verification of all (claim, chunk) pairs executes with low latency. High-throughput LLM adapters (via Groq API or quantized local Ollama instances) ensure reliable generation.

3.2 Operational Feasibility

VerifyRAG is designed with high operational usability. The end-user does not require specialized machine learning knowledge to interact with the system. Users upload standard documents (PDF, TXT) through a clean web interface and type queries in plain language. Every generated answer is augmented with color-coded verification badges and interactive citation pills. Single-click inspection allows users and evaluators to immediately examine the exact source passage that substantiated each claim.

3.3 Economic Feasibility

The project exhibits complete economic feasibility as it relies strictly on open-source software and open-access model weights available via Hugging Face. Development, indexing, and verification run locally on existing consumer hardware without requiring paid enterprise cloud clusters. External LLM querying utilizes free and ultra-low-cost API tiers (such as Groq or OpenAI gpt-4o-mini) or fully free offline local models, resulting in negligible financial expenditure.

3.4 Schedule Feasibility

The development lifecycle is divided into structured milestones matching the academic semester schedule: Month 1 focuses on document parsing, chunking, and FAISS vector indexing; Month 2 covers LLM draft generation, atomic claim splitting, and DeBERTa cross-encoder integration; Month 3 implements filtering, regeneration algorithms, and the Streamlit interface; Month 4 is dedicated to benchmark evaluation, comparative baseline analysis, and project documentation.

3.5 Legal Feasibility

VerifyRAG complies fully with legal and licensing standards. All underlying libraries (PyTorch, Hugging Face Transformers, FAISS, Streamlit) are licensed under permissive open-source licenses (MIT and Apache 2.0). All benchmark evaluation datasets and test documents utilize public-domain academic publications, honoring copyright laws, fair-use doctrines, and institutional academic ethics.

4. Methodology/ Planning of work

The engineering methodology of VerifyRAG is designed as a closed-loop verification architecture divided into four coordinated subsystems. The dataflow transitions from raw corpus indexing to atomic claim decomposition, batched cross-encoder inference, and dual-strategy remediation, as shown in Figure 1.

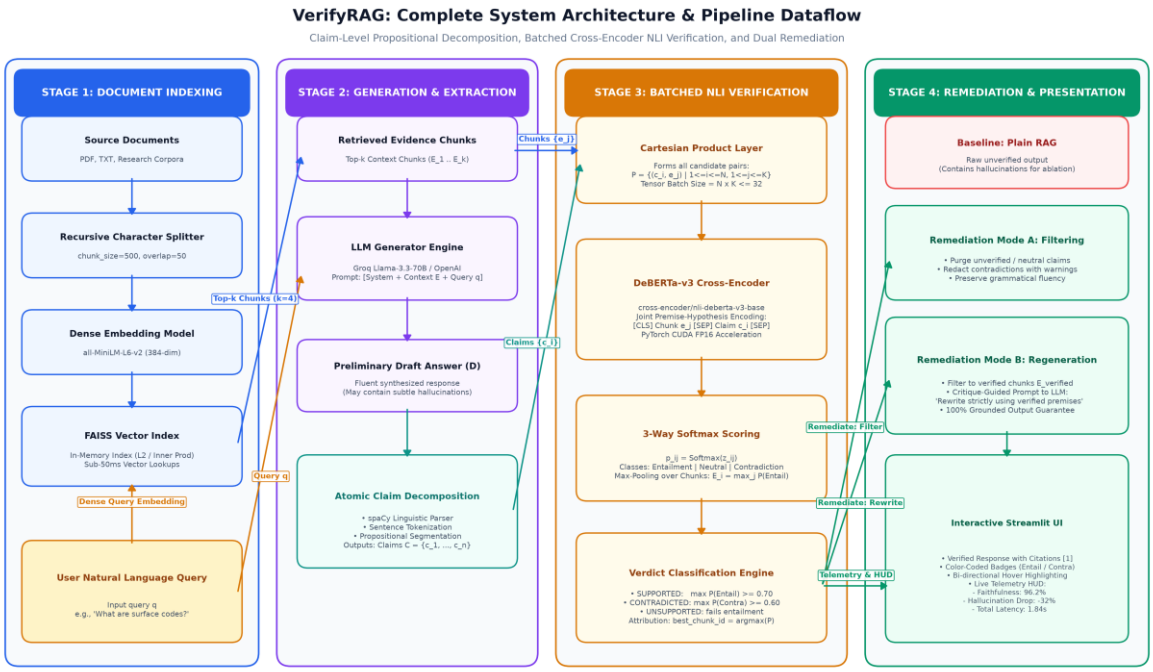


Figure 1: *End-to-end system architecture of VerifyRAG illustrating document indexing, generation, batched cross-encoder NLI verification, and dual remediation strategies.*

4.1 Subsystem 1: Document Indexing & Dense Semantic Retrieval

Unstructured knowledge documents (PDF, TXT, Markdown) are parsed and partitioned using recursive character boundary chunking with a segment length of 500 characters and a 50-character sliding overlap. Each text chunk e_j is projected into a 384-dimensional continuous latent space using the sentence-transformers/all-MiniLM-L6-v2 embedding model. The vectors are indexed using Facebook AI Similarity Search (FAISS) with inner-product cosine similarity. When a user submits query q, the vectorized query v_q performs a top-k nearest-neighbor search to retrieve the most semantically relevant evidence passages E = {e_1, e_2, ..., e_k} (default k=4) within sub-50ms latency.

4.2 Subsystem 2: Contextual Generation & Propositional Decomposition

The generator LLM (Groq Llama-3.3-70B or OpenAI GPT-4o-mini) receives an augmented prompt containing query q and evidence passages E to synthesize a preliminary draft answer D. Because standard LLM generation is prone to hallucination, D is not shown directly to the user. Instead, D is routed to the Claim Decomposition Engine, which uses spaCy's dependency parser and rule-based clausal segmentation to decompose compound sentences into a discrete set of atomic, independently testable propositions C = {c_1, c_2, ..., c_n}. Each claim encapsulates a single factual assertion.

4.3 Subsystem 3: Batched Cross-Encoder NLI Verification Engine

To fact-check claims against retrieved context with maximum semantic precision, VerifyRAG utilizes cross-encoder/nli-deberta-v3-base rather than bi-encoder cosine distance. A Cartesian candidate pairing layer constructs all candidate combinations P = {(c_i, e_j) | 1 <= i <= n, 1 <= j <= k}. For an answer of 5 claims and 4 retrieved chunks, the resulting 20 pairs are vectorized into a single batch (batch size <= 32) and evaluated concurrently on the GPU using PyTorch FP16 tensor operations.

The cross-encoder performs bidirectional joint cross-attention over sequence [CLS] e_j [SEP] c_i [SEP], generating logit vector z_ij in R^3. A Softmax transformation yields class probabilities for Contradiction, Neutral, and Entailment. For each claim c_i, max-pooling over all evidence chunks determines the strongest supporting chunk j* and peak entailment probability: E_i = max_j P(Entailment | e_j, c_i). A decision engine classifies the claim as SUPPORTED if E_i >= 0.70, CONTRADICTED if max_j P(Contradiction) >= 0.60, or UNSUPPORTED (Neutral) otherwise.

4.4 Subsystem 4: Dual Remediation & Attributed Presentation

Once claims are classified, VerifyRAG executes one of two remediation pathways before displaying the final response on the Streamlit dashboard:

- **Remediation Strategy A (Active Filtering):** Unsupported and neutral claims are systematically purged from the response text, while contradictory statements are either redacted or accompanied by inline warning callouts. The remaining supported claims are re-concatenated with grammatical transitions.
- **Remediation Strategy B (Critique-Guided Regeneration):** The system isolates strictly verified chunks E_verified and feeds a structured critique constraint back to the generator

LLM: 'Rewrite the answer using ONLY the verified facts from the provided sources; do not assume or introduce unverified details.' The LLM re-synthesizes a concise, 100% grounded response.

The final output renders on the web interface with interactive citation pills (e.g., [Chunk 2] (✓)), bi-directional hover highlighting connecting text to source chunks, and an automated Telemetry HUD tracking Faithfulness Score, Hallucination Suppression Rate, and component latency breakdown.

5. Tools/Technology Used:

5.1 Minimum Hardware Requirements

Hardware required for the development and execution of the project:

CPU: Intel Core i5 (11th / 12th Generation equivalent, Multi-core ≥ 2.5 GHz)

RAM: 16 GB DDR4 / DDR5

GPU: NVIDIA GeForce RTX 3050 (4 GB GDDR6 VRAM, CUDA Compute 8.6)

HDD/SSD: 512 GB NVMe Solid State Drive

Others (if any): Broadband Internet connection for model weight caching and API services

5.2 Minimum Software Requirements

Software and toolchain required for the development of the project:

OS: Microsoft Windows 11 (64-bit) / Ubuntu Linux 22.04 LTS

Programming Language: Python 3.10 / 3.11

Deep Learning Framework: PyTorch 2.2.0 with CUDA 12.1 acceleration

NLP & Transformers Hub: Hugging Face Transformers (v4.40+), Sentence-Transformers (v2.7+)

NLI Verification Model: cross-encoder/nli-deberta-v3-base

Vector Database: FAISS-CPU / FAISS-GPU (v1.8.0+)

Text Parsing Tools: spaCy (v3.7+, en_core_web_sm), PyPDF (v4.2+)

Web Presentation / Demo: Streamlit (v1.35+)

Development IDE: Antigravity IDE / Visual Studio Code

6. References: [IEEE format]:

The following study material and research publications were referenced for the development of this project:

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[2] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," in Proc. 9th Int. Conf. Learn. Represent. (ICLR), Virtual Event, 2021.

[3] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A Large-scale Dataset for Fact Extraction and VERification," in Proc. 2018 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT), New Orleans, LA, pp. 809–819, 2018.

[4] S. Min, K. Krishna, X. Lyu, M. Lewis, W. Yih, P. Kohli, S. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP), Singapore, pp. 12076–12100, 2023.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[6] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," in Thirty-fifth Conf. Neural Inf. Process. Syst. Datasets and Benchmarks Track, 2021.